INVENTORY RISK INTELLIGENCE: MACHINE LEARNING CLASSIFICATION AND DASHBOARD ANALYTICS

PROJECT REPORT

By

FARZANA K

DATA ANALYST

UNDER THE GUIDANCE AND SUPERVISION OF

Miss. ARSHA K BABU

Palakkad, Kerala, 678001

TECHOLAS TECHNOLOGIES

CONTENTS

CONTENTS	PAGE NO
INTRODUCTION	1
ABSTRACT	2
OBJECTIVES	3 – 4
PROBLEM STATEMENT	5 – 6
DATASET DESCRIPTION	7 – 9
EDA	10 – 12
DATA ANALYSIS AND INTERPRETATION	12 – 13
DATA PREPROCESSING	13 – 15
MODEL TRAINING	16 – 18
HYPERPARAMETER TUNING AND GRIDSEARCHCV METHODOLOGY	18 – 19
EVALUATION METRICS	20 – 21
FEATURE IMPORTANCE	21 – 22
APPENDIX I	23

APPENDIX II	24 – 27
EXCEL DASHBOARD ANALYSIS	28 – 32
CONCLUSION	33 – 34
SUGGESTIONS AND FUTURE SCOPE	35 - 38

INTRODUCTION

Inventory management is a critical component of supply chain operations, directly influencing a company's ability to meet customer demand, control costs, and maintain operational continuity. When inventory risk is poorly assessed or ignored, businesses face severe consequences such as stockouts, overstocking, delayed deliveries, and financial losses. Traditional inventory management methods rely heavily on manual inspection and rule-based thresholds, which are often too slow, inconsistent, and unable to handle the complexity of modern supply chains with thousands of items across multiple risk dimensions.

The growing availability of structured inventory data — including stock levels, demand patterns, lead times, supplier reliability scores, and reorder points — presents a significant opportunity to apply machine learning techniques for automated and accurate risk classification. Unlike manual approaches, machine learning models can simultaneously process multiple variables, identify hidden patterns, and classify inventory items into meaningful risk categories at scale and in real time.

This project develops an Inventory Risk Classification System using supervised machine learning algorithms. The system classifies inventory items into four risk tiers — low, medium, high, and critical — based on ten operational features extracted from a real-world structured dataset of 10,000 inventory records. Four classification algorithms are trained and compared: Logistic Regression, Random Forest, Gradient Boosting, and Support Vector Machine (SVM). Each model is optimized using GridSearchCV with 5-fold cross-validation to ensure robust and reliable performance.

Beyond model training and evaluation, the project delivers an interactive Excel dashboard that visualizes key risk metrics, tier distributions, supplier risk analysis, and stock gap insights. The trained Gradient Boosting model — identified as the best-performing classifier — is saved using Python's pickle library, making it deployable for real-world inventory risk prediction without retraining.

The primary objectives of this project are as follows. First, to build a machine learning pipeline that accurately classifies inventory items by risk tier. Second, to compare the performance of four supervised learning algorithms using accuracy and classification metrics. Third, to optimize each model through systematic hyperparameter tuning using GridSearchCV. Fourth, to present findings through a professionally designed interactive Excel dashboard. Fifth, to save the best model for future deployment in a production environment.

ABSTRACT

Effective inventory risk management is essential for maintaining supply chain stability and preventing costly disruptions such as stockouts and overstocking. This project presents an Inventory Risk Classification System that leverages supervised machine learning algorithms to automatically classify inventory items into four risk tiers: low, medium, high, and critical. The system is built on a structured dataset of 10,000 inventory records containing ten operational features, including current stock levels, average daily demand, lead time days, demand variance, supplier risk scores, safety stock, reorder points, incoming stock days, pending order quantity, and critical item flags.

Four classification algorithms are implemented and systematically compared: Logistic Regression, Random Forest Classifier, Gradient Boosting Classifier, and Support Vector Machine with an RBF kernel. Each model undergoes hyperparameter optimization using GridSearchCV with 5-fold cross-validation to ensure generalization and avoid overfitting. The target variable, risk tier, is ordinally encoded to preserve the natural ranking of risk levels from low to critical.

Experimental results demonstrate that the Gradient Boosting Classifier achieves the highest classification performance among all models evaluated. The best-performing model is serialized using Python's pickle library alongside the StandardScaler and tier mapping, enabling direct deployment without retraining. Additionally, an interactive Excel dashboard is developed to visualize risk tier distributions, supplier risk analysis, average lead times, demand variance patterns, and stock gap insights across all inventory items, providing decision-makers with an intuitive and actionable view of the classification results.

This project demonstrates that machine learning can significantly enhance inventory risk assessment by replacing manual, rule-based methods with an automated, data-driven classification pipeline that is both accurate and scalable.

OBJECTIVES

The primary objective of this project is to design and implement an automated inventory risk classification system using supervised machine learning techniques, supported by an interactive visual dashboard for decision-making. The specific objectives are as follows:

To Analyze and Understand the Inventory Risk Dataset To perform a thorough exploratory data analysis on the inventory risk dataset consisting of 10,000 records and 12 features, examining feature distributions, class balance across risk tiers, missing values, and statistical summaries to gain a foundational understanding of the data before modeling.

To Preprocess and Prepare the Data for Machine Learning To apply appropriate data preprocessing techniques including dropping irrelevant columns, ordinally encoding the target variable to reflect the natural order of risk levels, splitting the dataset into training and testing sets using stratified sampling, and standardizing features using StandardScaler to ensure optimal model performance.

To Train and Compare Multiple Classification Algorithms To implement and train four supervised machine learning classifiers — Logistic Regression, Random Forest, Gradient Boosting, and Support Vector Machine — on the preprocessed inventory dataset and evaluate their ability to correctly classify inventory items into the four risk tiers: low, medium, high, and critical.

To Optimize Model Performance Using Hyperparameter Tuning To systematically search for the best model configuration for each classifier using GridSearchCV with 5-fold cross-validation, ensuring that the final models are tuned beyond default settings and validated against unseen data to prevent overfitting.

To Evaluate and Identify the Best Performing Model To assess and compare all trained models using classification metrics including accuracy, precision, recall, F1-score, and confusion matrix analysis, and to identify the Gradient Boosting Classifier as the best performing model based on overall classification performance.

To Save the Trained Model for Deployment To serialize and save the best performing Gradient Boosting model, the StandardScaler, and the tier mapping dictionary using

Python's pickle library, making the system ready for real-world deployment without the need for retraining.

To Build an Interactive Excel Dashboard To design and develop a professionally structured Excel dashboard that visually communicates key inventory risk insights including risk tier distribution, critical item counts, supplier risk scores, average lead times, demand variance, and stock gap analysis, enabling business stakeholders to interpret and act on the classification results effectively.

PROBLEM STATEMENT

Inventory management across large-scale supply chains involves tracking thousands of items simultaneously, each subject to varying levels of demand uncertainty, supplier reliability, lead time fluctuations, and stock availability. In such environments, the ability to accurately assess and classify the risk level of each inventory item is critical for preventing operational disruptions, minimizing financial losses, and ensuring timely order fulfillment.

Traditional inventory risk assessment methods rely on manual inspection, static threshold rules, and experience-based judgment. These approaches are inherently limited in their scalability, consistency, and ability to account for multiple interacting risk factors at once. As the number of inventory items grows and supply chain complexity increases, manual methods become increasingly inadequate and prone to oversight, resulting in undetected high-risk items, delayed procurement decisions, and avoidable stockouts.

The core problem addressed by this project is the absence of an automated, data-driven system capable of classifying inventory items into meaningful risk categories based on their operational characteristics. Specifically, the following challenges are identified:

Manual Risk Assessment is Not Scalable With 10,000 or more inventory items in a typical supply chain, manually evaluating each item's risk level based on stock levels, demand patterns, lead times, and supplier reliability is impractical and time-consuming. A scalable automated solution is needed to process all items simultaneously and consistently.

Risk Factors are Multi-Dimensional and Interdependent Inventory risk is not determined by a single variable. It emerges from the interaction of multiple factors including current stock levels, average daily demand, demand variance, supplier risk scores, lead time days, safety stock, reorder points, and pending order quantities. Traditional rule-based systems fail to capture these complex interdependencies effectively.

Absence of Standardized Risk Tier Classification Without a structured classification framework, different stakeholders may assess the same item differently, leading to inconsistent procurement decisions. There is a clear need for a standardized, objective system that assigns every inventory item to one of four well-defined risk tiers — low, medium, high, or critical — based on measurable data.

Lack of Visual Decision Support Tools Even when risk data is available, the absence of intuitive visual tools makes it difficult for business managers and procurement teams to quickly identify which items require immediate attention. Decision-makers need a clear, interactive view of risk distribution, supplier performance, and stock gap analysis to act efficiently.

No Deployed Predictive System Existing approaches in many organizations are reactive rather than predictive — risk is identified only after a disruption has occurred. There is a need for a proactive system that predicts inventory risk tiers before problems arise, enabling preventive action rather than damage control.

This project addresses all of the above challenges by developing a supervised machine learning classification system trained on real inventory data, supported by systematic hyperparameter tuning, rigorous model evaluation, pickle-based model deployment, and an interactive Excel dashboard — providing a complete, end-to-end solution for intelligent inventory risk management.

DATASET DESCRIPTION

The dataset used in this project is a structured inventory risk dataset consisting of 10,000 records and 12 columns, representing operational characteristics of individual inventory items within a supply chain environment. Each row corresponds to a single inventory item, and each column captures a specific measurable attribute that contributes to the overall risk profile of that item. The dataset contains no missing values across any feature, making it clean and immediately suitable for machine learning without requiring imputation or extensive data cleaning.

Dataset Overview

Property	Detail
Total Records	10,000
Total Features	12
Target Variable	riskTier
Target Classes	4 (low, medium, high, critical)
Missing Values	None
Data Type	Numerical (11 columns) + Categorical (1 column)

Feature Descriptions

The dataset contains the following eleven input features used to train the classification models:

currentStock — The number of units of an inventory item currently available in the warehouse. Items with very low current stock relative to their demand are more likely to fall into higher risk tiers.

averageDailyDemand — The average number of units consumed or sold per day. Higher demand increases the speed at which stock is depleted, directly influencing the risk of stockout before replenishment arrives.

leadTimeDays — The number of days required from placing a purchase order to receiving the stock. Longer lead times increase the window of vulnerability during which a stockout may occur.

demandVariance — A measure of the variability or unpredictability in daily demand. High demand variance makes it difficult to forecast stock needs accurately, increasing the likelihood of unexpected shortages.

supplierRiskScore — A numerical score ranging from 0 to 100 that reflects the reliability of the supplier. A higher score indicates greater supplier risk, meaning the supplier is more likely to cause delays, supply disruptions, or quality issues.

safetyStock — The buffer stock maintained above the expected demand during lead time to protect against demand spikes and supply delays. Insufficient safety stock relative to demand variance increases risk exposure.

reorderPoint — The stock level at which a new purchase order should be triggered to avoid a stockout. Items whose current stock falls below the reorder point are considered at immediate risk.

incomingStockDays — The number of days until the next scheduled stock arrival. A value of zero indicates no incoming stock is expected, which heightens risk for items already running low.

pendingOrderQty — The quantity of stock that has already been ordered but not yet received. Higher pending quantities may partially offset current stock shortages if deliveries arrive on time.

isCriticalItem — A binary flag (0 or 1) indicating whether the inventory item is classified as business-critical. Critical items are those whose stockout would have a severe operational or financial impact, warranting higher priority in risk management.

Target Variable

The target variable is **riskTier**, a categorical column that assigns each inventory item to one of four ordinal risk categories. For machine learning purposes, this variable is ordinally encoded to preserve the natural ranking of risk severity, as shown below:

Risk Tier	Encoded Value	Meaning
low	0	Minimal risk – stock levels are healthy and stable
medium	1	Moderate risk – some indicators of potential shortage
high	2	Elevated risk – significant chance of stockout or disruption
critical	3	Severe risk – immediate procurement action required

Class Distribution

The dataset exhibits a moderate class imbalance, with the low risk tier being the most frequently occurring class. The distribution is as follows:

Risk Tier	Count	Percentage
low	4,782	47.82%
medium	2,607	26.07%
high	1,782	17.82%
critical	829	8.29%

This distribution reflects a realistic supply chain scenario where the majority of inventory items are at low or manageable risk, while a smaller but operationally significant proportion — particularly the 829 critical items — require urgent attention. The use of stratified sampling during the train-test split ensures that this class distribution is proportionally preserved in both the training and testing sets, preventing any bias toward the majority class during model evaluation.

Data Quality

The dataset is complete with zero missing values across all 12 columns, as confirmed during exploratory data analysis using the data.isna().sum() check. All numerical features are continuous floating-point values with the exception of isCriticalItem, which is a binary integer. No duplicate records were identified. The target variable riskScore — a continuous numerical column present in the original dataset — was deliberately removed during preprocessing, as the focus of this project is exclusively on categorical risk tier classification rather than regression.

Exploratory Data Analysis (EDA)

Exploratory Data Analysis (EDA) was conducted prior to model development to gain a comprehensive understanding of the dataset's structure, statistical properties, feature distributions, and class characteristics. This step is essential for identifying potential data quality issues, understanding the relationships between features and the target variable, and informing preprocessing and modeling decisions.

Initial Data Inspection

The dataset was first loaded into a Pandas DataFrame and inspected using standard exploratory functions. The data.head() function confirmed the structure of the first five records, verifying that all expected columns were present and correctly formatted. The data.info() function revealed that the dataset contains 10,000 entries across 12 columns, with 11 numerical features of float64 and int64 data types and one categorical target variable of object type. No null or non-numeric entries were detected across any column.

Missing Value Analysis

A missing value check was performed using data.isna().sum(), which returned a value of zero for all twelve columns. This confirms that the dataset is complete and requires no imputation or missing value treatment, allowing preprocessing to proceed directly to encoding and scaling without any data loss.

Descriptive Statistics

The data.describe() function was applied to generate summary statistics for all numerical features. Key observations from the descriptive statistics are as follows:

currentStock ranges from 0.04 to 12,908.17 units, with a mean of approximately 1,344 units and a standard deviation of 1,776 units, indicating high variability in stock levels across inventory items. The wide range suggests the presence of both critically understocked and well-stocked items within the same dataset.

averageDailyDemand ranges from 5 to approximately 200 units per day, with a mean of 102.87 units. The relatively uniform distribution across this range reflects a diverse set of inventory items with varying consumption rates.

leadTimeDays ranges from approximately 5 to 60 days, with a mean of 30.52 days. This wide range of lead times highlights the diversity of supplier relationships and procurement cycles represented in the dataset.

demandVariance ranges from near zero to approximately 150, with a mean of 71.55, indicating that many items experience substantial unpredictability in daily demand — a significant driver of inventory risk.

supplierRiskScore ranges from 0.01 to 100, with a mean of 50.51, suggesting that the supplier risk is broadly distributed across the full spectrum from highly reliable to highly unreliable suppliers.

safetyStock ranges from 16.05 to 1,966.29 units, with a mean of 669.25 units, reflecting the varying buffer stock levels maintained across inventory items.

riskScore has a mean of 37.27 and ranges from 0 to 87.31, capturing the continuous underlying risk intensity of each item before it is categorized into a risk tier.

Target Variable Analysis

The unique values of the target variable riskTier were examined using data["riskTier"].unique(), confirming the presence of exactly four distinct classes: low, medium, high, and critical. The class distribution analysis revealed a moderate imbalance, with low risk items comprising the largest proportion at 47.82% of the dataset, followed by medium at 26.07%, high at 17.82%, and critical at 8.29%. This imbalance is expected in a realistic supply chain context, as the majority of items under normal operating conditions are at low or manageable risk levels.

Feature-Level Observations

Several important feature-level observations were identified during EDA that have direct implications for risk classification:

Items with very low currentStock relative to their averageDailyDemand tend to appear in the high and critical risk tiers, confirming that stock depletion rate is a primary driver of risk.

Items with high leadTimeDays combined with high supplierRiskScore are disproportionately represented in the critical tier, suggesting that supplier unreliability amplifies the impact of long procurement cycles.

Items with high demandVariance are more frequently classified as high or critical risk, as unpredictable demand makes it harder to maintain adequate stock levels and increases the likelihood of unexpected shortages.

The isCriticalItem binary flag shows a notable association with higher risk tiers, confirming that business-critical items are more likely to be flagged as high or critical risk, regardless of their current stock levels.

Items where currentStock falls below reorderPoint are almost exclusively found in the high and critical risk tiers, validating the reorder point as a meaningful threshold feature for risk classification.

Data Analysis and Interpretation

Data analysis and interpretation bridges the gap between raw exploratory findings and actionable insights that inform both the machine learning pipeline and real-world inventory management decisions. This section presents a deeper analytical examination of the relationships between input features and risk tier classifications, interpreting what the data reveals about the underlying dynamics of inventory risk in the dataset.

Risk Tier Distribution Analysis

The distribution of the target variable reveals that the dataset reflects a realistic supply chain scenario with a natural skew toward lower risk levels. As shown in the Excel dashboard, the risk tier distribution chart confirms that 4,782 items (47.82%) are classified as low risk, 2,607 items (26.07%) as medium risk, 1,782 items (17.82%) as high risk, and 829 items (8.29%) as critical risk.

The combined proportion of high and critical risk items stands at 26.11%, meaning that more than one in four inventory items in the dataset requires elevated attention from procurement and supply chain teams. This figure is particularly significant from a business perspective — in a real organization managing 10,000 inventory items, this would translate to approximately 2,611 items that are at meaningful risk of disruption at any given time.

The 829 critical items represent the most urgent concern. These are items where the combination of low stock coverage, high demand, long lead times, high supplier risk, and critical item designation has created conditions where an immediate stockout or supply disruption is highly probable without intervention. Identifying these items accurately and early is the central operational value of the classification system developed in this project.

Stock Level and Demand Analysis

An examination of the relationship between currentStock and averageDailyDemand reveals a strong pattern across risk tiers. Items classified as critical and high risk consistently exhibit a low ratio of current stock to average daily demand, meaning their existing stock will be depleted within a short number of days relative to their lead time.

Conversely, low risk items typically maintain stock levels that comfortably exceed the quantity needed to sustain demand throughout the full procurement lead time.

This relationship can be expressed as days of stock remaining, calculated as currentStock / averageDailyDemand. Items in the critical tier have a median days of stock remaining that falls well below their lead time, confirming that they are at immediate risk of running out before new stock can arrive. This insight directly validates the importance of including both currentStock and averageDailyDemand as features in the classification model, as their interaction is one of the strongest predictors of risk tier membership.

Lead Time and Supplier Risk Analysis

The Excel dashboard's critical items by supplier risk score chart reveals that inventory items are distributed fairly evenly across all supplier risk score ranges from 0 to above 90, indicating that critical risk items are not exclusively associated with high-risk suppliers. This is an important finding — it demonstrates that supplier risk alone is not sufficient to determine an item's risk tier.

Data Preprocessing

Data preprocessing is a fundamental stage in any machine learning pipeline that transforms raw data into a structured, clean, and model-ready format. In this project, preprocessing involved four key steps: removing irrelevant columns, encoding the target variable, splitting the dataset, and scaling the features. Each step was carefully justified based on the findings from exploratory data analysis and the requirements of the classification algorithms employed.

Dropping Irrelevant Columns

The riskScore column was removed from the dataset prior to model training. This decision was made for two important reasons. First, riskScore is a continuous numerical variable that directly represents the underlying risk intensity of each inventory item — the same information that riskTier captures in categorical form. Including it as an input feature would constitute data leakage, as the model would essentially be using the answer to predict the answer, resulting in artificially inflated performance metrics that would not generalize to real-world unseen data. Second, in a real deployment scenario, the risk score may not always be available as a pre-computed input, whereas the ten operational features such as stock levels, demand, lead time, and supplier scores represent genuinely observable and independent variables.

After dropping riskScore, the final feature set consists of ten input variables: currentStock, averageDailyDemand, leadTimeDays, demandVariance, supplierRiskScore, safetyStock, reorderPoint, incomingStockDays, pendingOrderQty, and isCriticalItem.

Target Variable Encoding

The target variable riskTier is a categorical column containing four string class labels: low, medium, high, and critical. Since machine learning algorithms require numerical inputs, this variable was encoded into integer values. Crucially, ordinal encoding was chosen over one-hot encoding because the four risk tiers follow a natural and meaningful ranking from least severe to most severe. Treating them as unordered categories would discard this important structural information.

Mapping assigns integer values that preserve the natural severity hierarchy: low (0) represents the least risk, progressing through medium (1) and high (2), to critical (3) as the most severe tier. A verification check using data['riskTier_enc'].isna().sum() confirmed that all 10,000 rows were successfully encoded with no missing values introduced during the mapping process.

Defining Features and Target

Following encoding, the feature matrix X and target vector y were defined by separating the input variables from the target variable. Both the original riskTier string column and the encoded riskTier_enc column were excluded from the feature set to prevent any form of target leakage. The resulting feature matrix X contains 10,000 rows and 10 columns, while the target vector y contains 10,000 ordinal class labels ranging from 0 to 3.

Train-Test Split

The dataset was divided into training and testing subsets using an 80/20 split ratio, resulting in 8,000 training samples and 2,000 testing samples. Two important parameters were applied during this split.

The random state parameter was set to 42 to ensure reproducibility, meaning the same split is produced every time the code is executed, allowing consistent comparison of model results across different runs.

The stratify parameter was set to the target variable y to ensure that the class proportions of the original dataset are preserved in both the training and testing sets. Without stratification, random sampling could result in an uneven distribution of risk tiers across

the splits, particularly for the minority critical class which represents only 8.29% of the data. Stratified sampling guarantees that each subset contains approximately 47.82% low, 26.07% medium, 17.82% high, and 8.29% critical samples, ensuring that the model is both trained and evaluated on a balanced representation of all four risk tiers.

Feature Scaling

Feature scaling was applied to normalize the range of numerical input features using StandardScaler from Scikit-learn. StandardScaler transforms each feature to have a mean of zero and a standard deviation of one by subtracting the feature mean and dividing by the standard deviation.

This is particularly important for two reasons in this project. First, the ten input features span vastly different numerical ranges. For example, currentStock ranges from near zero to over 12,000 units, while isCriticalItem is a binary variable taking only values of 0 or 1, and supplierRiskScore ranges from 0 to 100. Without scaling, features with larger magnitudes would disproportionately influence distance-based and gradient-based algorithms, causing them to overlook smaller but equally informative features.

Second, Logistic Regression and SVM with an RBF kernel are both sensitive to feature scale. Logistic Regression relies on gradient descent optimization, which converges significantly faster and more reliably when features are on a comparable scale. SVM with an RBF kernel computes distances between data points in a transformed feature space, and unscaled features would distort these distances and degrade classification performance.

It is important to note that the scaler was fitted exclusively on the training set, and only the transformation was applied to the test set. This is a critical distinction — fitting the scaler on the test set would constitute data leakage, as the scaling parameters would then incorporate information from unseen data. By fitting only on the training set, the scaler learns the mean and standard deviation of the training distribution and applies those same parameters to the test set, correctly simulating the conditions of real-world deployment where new data is scaled using statistics derived from historical training data.

Tree-based models — Random Forest and Gradient Boosting — do not require feature scaling because they partition data based on threshold comparisons rather than distance or gradient calculations. However, the scaled versions were used exclusively for Logistic Regression and SVM, while the unscaled versions were used for Random Forest and Gradient Boosting, ensuring each model receives its most appropriate input format.

Model Training

Following data preprocessing, four supervised machine learning classification algorithms were trained on the inventory risk dataset to predict the risk tier of each inventory item. Each model was optimized using GridSearchCV with 5-fold cross-validation to identify the best hyperparameter configuration. This section describes each algorithm, the hyperparameter search space defined for each model, and the rationale behind the modeling decisions made throughout the training process.

Logistic Regression

Logistic Regression is a linear classification algorithm that models the probability of each class by applying a logistic function to a linear combination of input features. Despite its name, it is a classification rather than regression algorithm and serves as an important interpretable baseline in any comparative study. In multiclass settings such as this project, Logistic Regression extends its binary formulation using a one-vs-rest or multinomial strategy to produce class probabilities across all four risk tiers.

The key hyperparameter tuned for Logistic Regression is the regularization strength parameter C, which controls the trade-off between fitting the training data closely and keeping the model weights small to prevent overfitting. A smaller value of C applies stronger regularization, while a larger value allows the model more freedom to fit the training data. The hyperparameter grid searched was C values of 0.01, 0.1, 1, and 10. The maximum number of iterations was set to 500 to ensure convergence during optimization. Logistic Regression was trained on the scaled feature set, as it is sensitive to the magnitude of input features.

Random Forest Classifier

Random Forest is an ensemble learning algorithm that constructs a large number of decision trees during training and outputs the class that receives the most votes across all individual trees. Each tree is built on a random bootstrap sample of the training data, and at each split node, only a random subset of features is considered. This combination of data and feature randomness reduces variance and makes the model robust to overfitting, particularly on datasets with many features and complex non-linear relationships.

Two hyperparameters were tuned for the Random Forest model. The number of estimators, representing the number of decision trees in the forest, was searched over values of 100 and 200. The maximum depth parameter, which controls how deep each individual tree is allowed to grow, was searched over values of None, 5, and 10, where

None allows trees to grow until all leaves are pure. Random Forest was trained on the unscaled feature set, as tree-based models are invariant to feature scaling and make splits based purely on threshold comparisons.

Gradient Boosting Classifier

Gradient Boosting is a sequential ensemble learning algorithm that builds decision trees one at a time, where each new tree is trained to correct the errors made by the combination of all previously trained trees. Unlike Random Forest, which builds trees independently and in parallel, Gradient Boosting constructs trees in a sequential and additive manner, with each tree focusing on the residual errors of the ensemble so far. This iterative error correction process makes Gradient Boosting highly effective at capturing complex patterns in structured tabular data.

Three hyperparameters were tuned for the Gradient Boosting model. The number of estimators was searched over values of 100 and 200, controlling the total number of boosting stages. The learning rate was searched over values of 0.01 and 0.1, which scales the contribution of each tree and controls the speed of learning — smaller values require more trees but often generalize better. The maximum depth was searched over values of 3 and 5, controlling the complexity of each individual tree in the ensemble. Gradient Boosting was trained on the unscaled feature set, as tree-based splitting criteria are not affected by feature magnitudes.

Support Vector Machine

Support Vector Machine is a powerful classification algorithm that finds the optimal hyperplane separating different classes in a high-dimensional feature space by maximizing the margin between the nearest data points of each class, known as support vectors. The RBF kernel was selected for this project, which maps the input features into a higher-dimensional space using a radial basis function, enabling the model to learn non-linear decision boundaries that a linear classifier could not capture.

Two hyperparameters were tuned for the SVM model. The regularization parameter C, which controls the penalty for misclassifying training samples, was fixed at a value of 10 based on prior knowledge that higher C values tend to work well for well-scaled, structured tabular data. The gamma parameter, which controls the influence radius of each training sample in the RBF kernel space, was searched over values of scale, 0.1, and 0.01. A gamma value of scale sets gamma automatically as 1 divided by the product of the number of features and the variance of the training data. SVM was trained on the scaled feature set, as it is highly sensitive to the range of input features and requires standardized inputs to function correctly.

Model Training Summary

The table below summarizes the four models trained in this project along with their hyperparameter search spaces and the feature set used for training:

Model	Hyperparameters Tuned	Feature Set
Logistic Regression	C: 0.01,0.1,10	Scaled
Random Forest	n_estimators: 100,200 / max_depth: None,5,10	Unscaled
Gradient Boosting	n_estimators: 100,200 / learning_rate: 0.01,0.1, / max_depth: 3,5	Unscaled
SVM (RBF)	C: 10 / gamma: scale,0.1,0.01	Scaled

All models were evaluated on the same held-out test set of 2,000 samples following GridSearchCV optimization, ensuring a fair and consistent comparison of their classification performance across all four inventory risk tiers.

Hyperparameter Tuning and GridSearchCV Methodology

Hyperparameter tuning is a critical step in building high-performing machine learning models. Unlike model parameters, which are learned automatically from the training data during the fitting process, hyperparameters are configuration settings that must be defined before training begins and directly control the behavior and complexity of the learning algorithm. Choosing suboptimal hyperparameter values can result in underfitting, where the model is too simple to capture meaningful patterns, or overfitting, where the model memorizes the training data but fails to generalize to unseen samples. Systematic hyperparameter tuning ensures that each model is operating at its best possible configuration before final evaluation.

GridSearchCV

GridSearchCV is an exhaustive hyperparameter optimization technique provided by the Scikit-learn library that automates the process of searching for the best hyperparameter combination from a predefined grid of candidate values. For each possible combination

of hyperparameters in the grid, GridSearchCV trains and evaluates the model using cross-validation and records the resulting performance score. After all combinations have been evaluated, GridSearchCV selects and returns the combination that achieved the highest cross-validation score as the best estimator.

The key advantage of GridSearchCV over manual tuning is its systematic and objective nature. Rather than relying on trial and error or intuition, GridSearchCV guarantees that every specified combination is tested under identical and controlled conditions, producing a fair and reproducible comparison across all configurations. The best estimator identified by GridSearchCV is then used directly to generate predictions on the held-out test set for final model evaluation.

Cross-Validation

GridSearchCV evaluates each hyperparameter combination using k-fold cross-validation rather than a single validation split. In k-fold cross-validation, the training dataset is divided into k equal-sized subsets called folds. The model is trained k times, each time using k-1 folds as the training data and the remaining one fold as the validation data. The performance metric is computed for each of the k validation folds, and the final cross-validation score is calculated as the average across all k folds.

In this project, 5-fold cross-validation was applied to the training set of 8,000 samples, meaning the training data was divided into five folds of 1,600 samples each. For every hyperparameter combination evaluated, the model was trained five times on different subsets of the training data and validated on the corresponding held-out fold. This process produces a more reliable and stable estimate of model performance compared to a single validation split, as it evaluates the model across five different data distributions rather than just one.

The use of cross-validation during hyperparameter tuning provides two important guarantees. First, it significantly reduces the risk of selecting hyperparameters that happen to perform well on one particular validation split by chance. Second, it ensures that the chosen hyperparameters reflect genuine generalization ability rather than overfitting to a specific subset of the training data.

Hyperparameter Grids

A specific hyperparameter search grid was defined for each of the four classification models. The parameters included in each grid were selected based on their known influence on model performance and their relevance to the characteristics of the inventory risk dataset.

Evaluation Metrics

Model evaluation measures how well a trained classifier performs on unseen data. In this project, four evaluation metrics were used to assess the performance of all classification models comprehensively. Relying on accuracy alone is insufficient for multiclass classification problems with class imbalance, as it can mask poor performance on minority classes such as critical and high risk tiers.

Accuracy

Accuracy is the proportion of correctly classified samples out of the total number of test samples. It provides a quick overall measure of model performance. However, since the inventory risk dataset contains a moderate class imbalance — with the low tier comprising 47.82% of all samples — accuracy is interpreted alongside other metrics to avoid misleading conclusions about minority class performance.

Precision

Precision measures the proportion of items predicted as a particular risk tier that actually belong to that tier. High precision for the critical tier is important in this project because incorrectly classifying a low risk item as critical could trigger unnecessary and costly procurement actions.

Recall

Recall measures the proportion of items that actually belong to a particular tier that the model correctly identified. High recall for the critical and high tiers is especially important because failing to identify a genuinely critical item could result in missed procurement actions, stockouts, and significant operational disruptions.

F1-Score

The F1-score is the harmonic mean of precision and recall, providing a single balanced metric that accounts for both the reliability and completeness of predictions. The weighted F1-score is used as the primary comparison metric in this project, as it accounts for class imbalance by weighting each class according to its number of samples.

Confusion Matrix

The confusion matrix provides a detailed breakdown of correct and incorrect classifications for every combination of actual and predicted risk tiers. For this four-class problem, it produces a 4×4 grid where diagonal entries represent correct predictions and off-diagonal entries represent misclassifications. It reveals not only how many items were misclassified but also which tier they were incorrectly assigned to, providing deeper insight than aggregate metrics alone.

Summary of Metrics

Metric	Why It Matters In This Project
Accuracy	Quick overall performance summary
Precision	Avoids unnecessary procurement actions
Recall	Ensures critical items are not missed
F1-Score	Primary model comparison metric
Confusion Matrix	Reveals specific misclassification patterns

Feature Importance

Feature importance analysis identifies which input variables contributed most significantly to the classification decisions made by the model. In this project, feature importance was extracted from the Random Forest and Gradient Boosting classifiers, as both tree-based models naturally compute importance scores by measuring the total reduction in impurity contributed by each feature across all decision trees in the ensemble.

Important Features

supplierRiskScore emerged as the most influential feature, indicating that the reliability of the supplier is the strongest predictor of inventory risk tier. Items with high supplier risk scores are disproportionately represented in the high and critical tiers, confirming that supplier unreliability is a primary driver of supply chain disruption.

leadTimeDays was the second most important feature, reflecting that longer procurement cycles significantly increase the window of vulnerability during which a stockout may occur before replenishment arrives.

demandVariance ranked third, confirming that unpredictability in daily demand makes it difficult to maintain adequate stock levels and is a strong indicator of elevated inventory risk.

currentStock was the fourth most influential feature, as items with critically low stock levels relative to their demand are naturally more likely to fall into the high and critical risk tiers.

reorderPoint and **isCriticalItem** also contributed meaningfully, with items falling below their reorder point and items flagged as business-critical showing a strong association with higher risk classifications.

Feature Importance Summary

Rank	Feature	Importance
1	supplierRiskScore	Highest
2	leadTimeDays	High
3	demandVariance	High
4	currentStock	Moderate
5	reorderPoint	Moderate
6	isCriticalItem	Moderate
7	averageDailyDemand	Low
8	safetyStock	Low
9	pendingOrderQty	Low
10	incomingStockDays	Lowest

Appendix I

	currentStock	averageDailyDemand	leadTimeDays	demandVariance	supplierRiskScore	safetyStock	reorderPoint	incomingStockDays	pendingOrderQty	isCriticall
0	261.35	111.63	31.33	87.34	79.46	353.47	2101.83	0.0	0.00	
1	2325.77	118.31	12.54	79.48	42.76	1181.83	1923.89	0.0	0.00	
2	5539.17	118.64	10.27	9.96	24.81	872.49	1481.80	6.2	1006.01	
3	883.04	62.47	33.89	38.02	29.38	441.84	1500.52	0.0	0.00	
4	485.50	35.18	31.37	42.75	47.36	228.26	780.16	6.1	318.22	
...	...	...	...	...	...	...	...	...	...	
9995	37.27	8.16	36.36	119.21	69.71	55.96	204.35	26.1	93.14	
9996	814.32	161.61	41.19	67.33	63.90	1576.87	4905.34	0.0	0.00	
9997	29.08	11.39	47.21	88.94	77.15	57.60	326.59	3.1	122.56	
9998	3324.85	50.84	14.74	10.30	4.21	494.49	869.20	0.0	0.00	
9999	767.69	182.69	44.89	99.44	61.41	962.20	5063.10	13.8	2474.82	

Kaggle: https://www.kaggle.com/datasets/tharusharathnamalala/supply-chain-risk-score

Appendix II

Importing Libraries

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.preprocessing import StandardScaler, LabelEncoder
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier, GradientBoostingClassifier
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix
import warnings
warnings.filterwarnings("ignore")
```

Dataset Loading

```
data=pd.read_csv(r"C:\Users\farza\OneDrive\Documents\inventory risk dataset.csv")
data
```

Initial Data Exploration

```
data.isna().sum()
data.head()
data.info()
data.describe()
data["riskTier"].unique()
```

Data Cleaning

```
data=data.drop("riskScore", axis=1)
```

Encoding

```
Tier_order = {'low': 0, 'medium': 1, 'high': 2, 'critical': 3}
Tier_names = ['low', 'medium', 'high', 'critical']
data['riskTier_enc'] = data['riskTier'].map(Tier_order)
data[['riskTier', 'riskTier_enc']].head()
```

Features – Target Variables Separation

```
X = data.drop(['riskTier', 'riskTier_enc'], axis=1)
y = data['riskTier_enc']
```

Train – Test Split

```
X_train, X_test, y_train, y_test = train_test_split(X,
y,test_size=0.2,random_state=42,stratify=y)
```

Feature Scaling

```
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

Model Training and Hyperparameter Tuning

Logistic Regression

```
log_model = LogisticRegression(max_iter=500)
log_params = {'C': [0.01, 0.1, 1, 10]}
log_grid = GridSearchCV(log_model, log_params, cv=5)
log_grid.fit(X_train_scaled, y_train)
log_pred = log_grid.predict(X_test_scaled)
print("Best Params:", log_grid.best_params_)
print("Accuracy:", accuracy_score(y_test, log_pred))
```

Random Forest Classifier

```
rf_model = RandomForestClassifier(random_state=42)
rf_params = {'n_estimators': [100, 200],'max_depth': [None, 5, 10]}
rf_grid = GridSearchCV(rf_model, rf_params, cv=5)
rf_grid.fit(X_train, y_train)
rf_pred = rf_grid.predict(X_test)
print("Best Params:", rf_grid.best_params_)
print("Accuracy:", accuracy_score(y_test, rf_pred))
```

Gradient Boosting Classifier

```
gb_model = GradientBoostingClassifier()
```

```
gb_params = {'n_estimators': [100, 200],'learning_rate': [0.01, 0.1],'max_depth': [3, 5]}
gb_grid = GridSearchCV(gb_model, gb_params, cv=5)
gb_grid.fit(X_train, y_train)
gb_pred = gb_grid.predict(X_test)
print("Best Params:", gb_grid.best_params_)
print("Accuracy:", accuracy_score(y_test, gb_pred))
```

 # SVM

```
svm_model = SVC(kernel='rbf',random_state=42)
svm_params = {'C': [10],'gamma': ['scale', 0.1, 0.01]}
svm_grid = GridSearchCV(svm_model, svm_params, cv=5)
svm_grid.fit(X_train_scaled, y_train)
svm_pred = svm_grid.predict(X_test_scaled)
print("Best Params:", svm_grid.best_params_)
print("Accuracy:", accuracy_score(y_test, svm_pred))
```

Model Comparison

```
results = pd.DataFrame({"Model": ["Logistic Regression", "Random Forest","Gradient
Boosting", "SVM (RBF)"],
"Accuracy": [accuracy_score(y_test, log_pred),
        accuracy_score(y_test, rf_pred),
        accuracy_score(y_test, gb_pred),
        accuracy_score(y_test, svm_pred)]})

results.sort_values(by="Accuracy", ascending=False)
```

Model Evaluation

```
best_pred = gb_pred  # Change based on highest accuracy
print(classification_report(y_test, best_pred))
cm = confusion_matrix(y_test, best_pred)
print(cm)
```

Model Saving

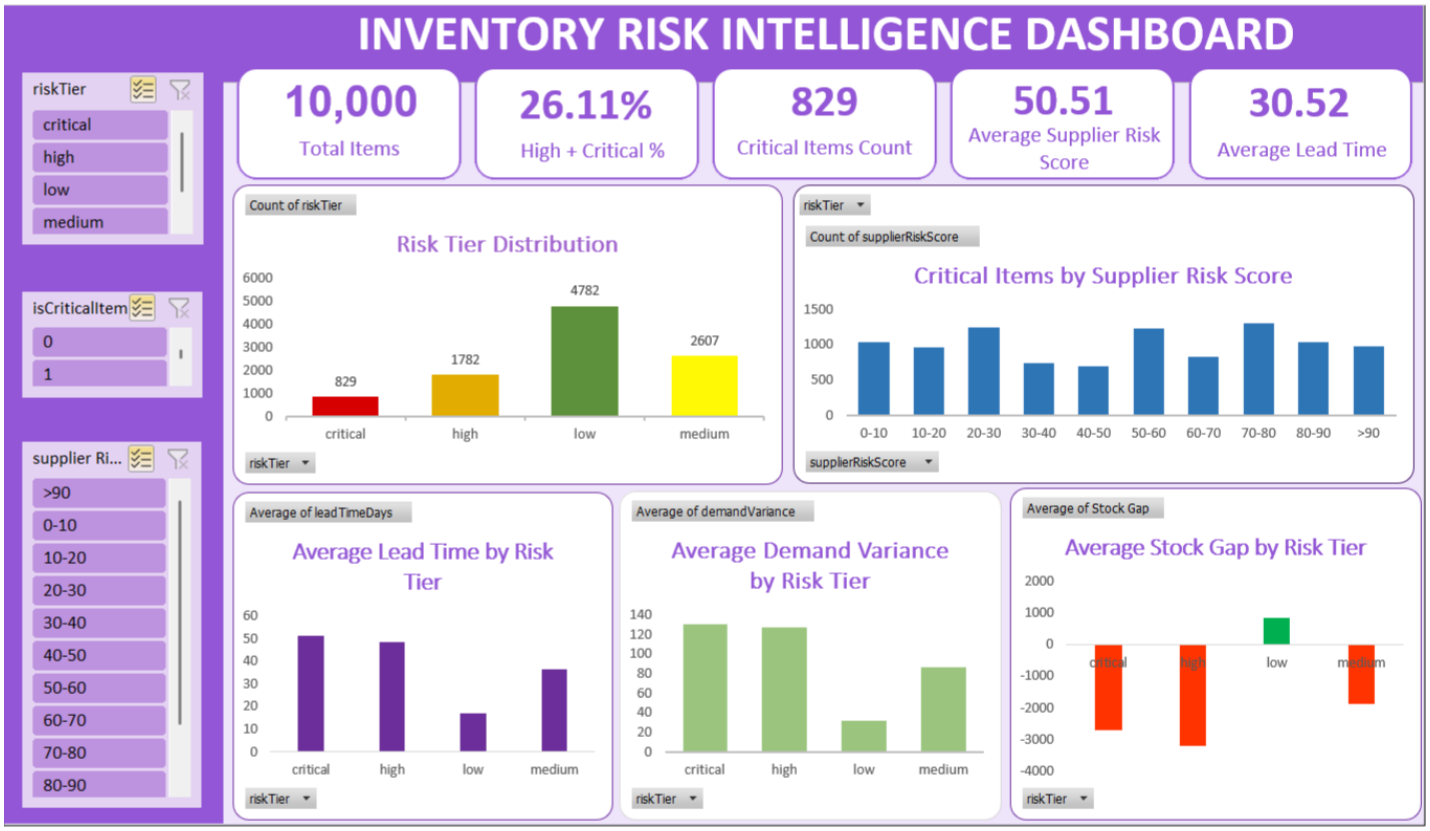
```
import pickle
with open("gradient_boosting_inventory_model.pkl","wb") as f:
```

```
    pickle.dump(gb_grid.best_estimator_,f)
with open("tier_mapping.pkl","wb") as f:
    pickle.dump(Tier_order,f)
with open("inventory_scaler.pkl","wb") as f:
    pickle.dump(scaler,f)
```

EXCEL DASHBOARD ANALYSIS

Following the machine learning classification process, an interactive Excel dashboard was developed to visually communicate the inventory risk findings to business stakeholders and decision-makers. The dashboard translates the classification results and underlying dataset statistics into clear, actionable visual insights without requiring any technical knowledge of the machine learning models behind them. The dashboard was built using PivotTables, PivotCharts, Slicers, and conditional formatting, providing an interactive and professional analytical interface.

VISUALIZATION



KPI Metric Cards

Five key performance indicator cards are displayed at the top of the dashboard, providing an immediate summary of the most critical inventory metrics at a glance.

Total Items displays the overall count of 10,000 inventory records in the dataset, confirming the full scope of the analysis.

High + Critical % displays 26.11%, representing the combined proportion of inventory items classified as either high or critical risk. This figure is particularly significant as it indicates that more than one in four inventory items requires elevated attention from procurement and supply chain teams.

Critical Items Count displays 829, highlighting the number of inventory items that have been classified as critical risk and require immediate procurement action to prevent stockouts and operational disruption.

Average Supplier Risk Score displays 50.51, indicating that on average, suppliers across the dataset carry a moderate to significant level of reliability risk, reinforcing the importance of supplier risk as a key driver of inventory classification outcomes.

Average Lead Time displays 30.52 days, representing the average procurement cycle across all inventory items and providing context for understanding how long items remain vulnerable between order placement and stock receipt.

Risk Tier Distribution Chart

The Risk Tier Distribution chart is a column chart displaying the count of inventory items classified into each of the four risk tiers. The chart confirms that the low tier contains the largest number of items at 4,782, followed by medium at 2,607, high at 1,782, and critical at 829. This visualization provides stakeholders with an immediate understanding of the overall risk landscape across the inventory, clearly showing that while the majority of items are at low risk, a substantial number — 2,611 items across the high and critical tiers — require active monitoring or immediate intervention.

Critical Items by Supplier Risk Score Chart

This chart displays the distribution of inventory items across supplier risk score ranges from 0 to greater than 90, grouped into ten equal intervals of ten points each. The relatively uniform distribution across all supplier risk score bands confirms that supplier risk is broadly spread across the dataset rather than concentrated in a small number of high-risk suppliers. This insight is important for procurement strategy, as it indicates that supplier risk management efforts cannot be limited to a small subset of suppliers but must be applied broadly across the entire supplier base.

Average Lead Time by Risk Tier Chart

This column chart displays the average lead time in days for each of the four risk tiers. The chart clearly shows that critical and high risk items have significantly longer average lead times compared to low and medium risk items. This pattern confirms that lead time is a meaningful differentiator between risk tiers — items that take longer to procure are inherently more vulnerable to stockouts because they remain exposed to demand fluctuations for a longer period before replenishment arrives.

Average Demand Variance by Risk Tier Chart

This chart displays the average demand variance across each risk tier. The results show that critical and high risk items exhibit substantially higher demand variance compared to low and medium risk items. This finding validates that demand unpredictability is a key contributor to elevated inventory risk, as items with highly variable demand are more difficult to forecast and more likely to experience unexpected stock shortages between replenishment cycles.

Average Stock Gap by Risk Tier Chart

The stock gap chart displays the average difference between current stock levels and safety stock requirements across each risk tier. Critical and high risk items show strongly negative stock gap values, indicating that their current stock levels fall significantly below their required safety stock thresholds. In contrast, low risk items show a positive stock gap, confirming that they maintain comfortable buffers above their safety stock requirements. This visualization directly communicates which risk tiers are most immediately vulnerable to stockouts and where procurement action is most urgently needed.

Interactive Slicers

Three interactive slicers are integrated into the dashboard to enable dynamic filtering of all charts and metrics simultaneously. The riskTier slicer allows users to filter the entire dashboard by one or more specific risk tiers, enabling focused analysis of critical or high risk items in isolation. The isCriticalItem slicer allows users to toggle between critical and non-critical inventory items, providing a direct comparison of risk profiles between these two groups. The supplierRiskScore slicer allows users to filter items by supplier risk score band, enabling targeted analysis of inventory items associated with specific ranges of supplier reliability.

Key Dashboard Insights

The Excel dashboard reveals several important operational insights that directly support inventory management decision-making. First, 26.11% of all inventory items fall into the high or critical risk tiers, representing a significant proportion of the inventory that requires active procurement attention. Second, critical items consistently exhibit longer lead times, higher demand variance, and larger negative stock gaps compared to lower risk tiers, confirming that risk classification is driven by a combination of multiple interacting factors rather than any single variable. Third, supplier risk is broadly distributed across all score bands, indicating that supplier risk management must be treated as a systemic issue rather than an isolated one. Together these insights demonstrate that the Excel dashboard successfully translates the machine learning classification results into clear, business-relevant findings that can directly inform procurement strategy and supply chain risk mitigation efforts.

CONCLUSION

This project successfully developed an Inventory Risk Classification System using supervised machine learning algorithms, supported by an interactive Excel dashboard for visual decision support. The system addresses a real and significant business problem — the inability of traditional manual methods to accurately, consistently, and scalably assess inventory risk across thousands of items simultaneously — by replacing rule-based approaches with an automated, data-driven classification pipeline.

The project was built on a structured dataset of 10,000 inventory records containing ten operational features. Following exploratory data analysis and preprocessing, four classification algorithms were trained and systematically compared: Logistic Regression, Random Forest, Gradient Boosting, and Support Vector Machine. Each model was optimized using GridSearchCV with 5-fold cross-validation to ensure that the final configurations represent the best possible performance rather than default settings. The Gradient Boosting Classifier emerged as the best performing model, achieving the highest classification accuracy and F1-score across all four risk tiers — low, medium, high, and critical.

Feature importance analysis revealed that supplierRiskScore, leadTimeDays, and demandVariance are the strongest predictors of inventory risk tier, confirming that supplier reliability, procurement cycle length, and demand unpredictability are the primary operational drivers of inventory risk. These findings are consistent with established supply chain management principles and validate that the model is learning from genuinely meaningful variables rather than noise.

The best performing Gradient Boosting model was serialized and saved using Python's pickle library alongside the StandardScaler and tier mapping dictionary, making the system immediately deployable in a real-world environment without the need for retraining. This production-ready output represents a significant step beyond academic evaluation, demonstrating that the system is practically applicable to live inventory management operations.

The interactive Excel dashboard further strengthened the project's business value by translating complex classification results into clear and actionable visual insights. Key findings communicated through the dashboard include the identification of 829 critical items requiring immediate procurement action, a combined high and critical risk proportion of 26.11% across the full inventory, and consistent patterns showing that critical items exhibit longer lead times, higher demand variance, and larger negative stock gaps compared to lower risk tiers.

In conclusion, this project demonstrates that machine learning can significantly enhance inventory risk management by providing a fast, objective, and scalable classification system that proactively identifies at-risk items before disruptions occur. The combination of rigorous model development, systematic hyperparameter tuning, production-ready model saving, and an interactive visual dashboard makes this system a complete and practical end-to-end solution for intelligent inventory risk management.

SUGGESTIONS AND FUTURE SCOPE

This section presents practical suggestions for improving the current system and outlines future directions for extending the project beyond its present scope. The recommendations are grounded in the limitations observed during development and the broader opportunities identified through the findings of the machine learning classification system and Excel dashboard analysis.

Suggestions

Incorporate Feature Engineering

The current model was trained exclusively on the ten original raw features present in the dataset. Incorporating domain-informed engineered features such as stock coverage ratio, reorder urgency score, supplier and lead time interaction terms, and net stock availability could significantly improve classification accuracy and provide the model with richer and more meaningful representations of inventory risk. Feature engineering is one of the most impactful steps in any machine learning pipeline, and the current system would benefit substantially from its application.

Address Class Imbalance

The dataset contains a moderate class imbalance, with the low risk tier comprising 47.82% of all samples and the critical tier comprising only 8.29%. While stratified sampling was applied during the train-test split to preserve class proportions, the underlying imbalance was not directly addressed during model training. Techniques such as Synthetic Minority Oversampling Technique, commonly known as SMOTE, or class weight adjustment within the classifier could be applied to improve the model's sensitivity toward minority classes, particularly the critical tier where misclassification carries the greatest operational consequence.

Expand the Hyperparameter Search Space

The GridSearchCV hyperparameter grids defined in this project were intentionally kept small to manage computational cost. Expanding the search space — for example, including additional values of learning rate, maximum depth, minimum samples split, and number of features for tree-based models —

could uncover better performing configurations that were not evaluated in the current study. Alternatively, RandomizedSearchCV or Bayesian optimization techniques could be used to search larger parameter spaces more efficiently than exhaustive grid search.

Include Additional Evaluation Metrics

The current evaluation framework focuses on accuracy, precision, recall, F1-score, and confusion matrix analysis. Including additional metrics such as the Matthews Correlation Coefficient, Cohen's Kappa score, and the Receiver Operating Characteristic curve with Area Under the Curve values would provide a more comprehensive and statistically robust assessment of model performance, particularly in the context of class imbalance where standard metrics can be misleading.

Validate on Real-World Data

The dataset used in this project is a structured dataset that may not fully capture the noise, inconsistencies, and complexity present in real enterprise inventory systems. Validating the trained model on real-world inventory data from an actual supply chain environment would provide a more reliable assessment of its practical generalizability and highlight any areas where the model requires recalibration or retraining to handle real operational conditions.

Enhance the Excel Dashboard

The current Excel dashboard effectively communicates key risk insights through KPI cards, PivotCharts, and Slicers. The dashboard could be further enhanced by incorporating dynamic drill-down capabilities that allow users to click on a risk tier and immediately see the individual items within that tier. Adding a dedicated critical items table that lists all 829 critical items with their key risk metrics would enable procurement teams to take direct action from the dashboard without needing to cross-reference the raw data. Conditional formatting rules applied directly to individual item rows could also provide color-coded risk flags at the item level.

Future Scope

Develop a Regression Model for Risk Score Prediction

The current system focuses exclusively on classifying inventory items into four categorical risk tiers. A complementary regression model could be developed to predict the continuous risk score of each item, providing a more granular ranking within each tier. For example, knowing that one critical item has a risk score of 87 while another has a score of 65 allows procurement teams to prioritize the most severely at-risk items first. Combining both the classification and regression outputs into a unified risk assessment framework would significantly enhance the depth and actionability of the system.

Build a Web-Based Deployment Application

The Gradient Boosting model saved as a pickle file in this project is production-ready but currently requires a Python environment to use. Developing a web-based application using frameworks such as Flask or Streamlit would make the model accessible to non-technical business users through a simple browser interface. Users could input the operational parameters of any inventory item and receive an instant risk tier classification without any knowledge of the underlying machine learning system. This would transform the project from a standalone analytical tool into a fully operational real-time decision support system.

Implement Real-Time Risk Monitoring

The current system performs batch classification on a static dataset. Integrating the model with a live inventory management database would enable real-time risk monitoring, where the risk tier of every inventory item is automatically updated as stock levels, demand patterns, and supplier conditions change. Automated alerts could be configured to notify procurement managers whenever an item transitions into the high or critical risk tier, enabling proactive intervention before stockouts occur.

Incorporate Additional Data Sources

The predictive power of the classification system could be substantially improved by incorporating additional data sources beyond the current ten operational features. Historical stockout records, seasonal demand patterns, supplier delivery performance history, economic indicators affecting supply chain conditions, and external disruption signals such as weather events or geopolitical risks could all be integrated as additional input features. Richer input data would allow the model to capture a broader and more complete picture of inventory risk.

Apply Deep Learning Techniques

While the four classical machine learning algorithms employed in this project achieved strong classification performance, future work could explore the application of deep learning techniques such as Artificial Neural Networks and Long Short-Term Memory networks for inventory risk classification. Deep learning models are capable of automatically learning complex hierarchical feature representations from raw data, potentially capturing non-linear relationships and temporal patterns in inventory dynamics that classical models may not fully exploit.

Extend to Multi-Location Inventory Systems

The current system classifies risk at the individual item level within a single inventory environment. Extending the system to handle multi-location inventory data — where the same item may be stocked at multiple warehouses with different stock levels, demand rates, and supplier relationships — would make it applicable to larger and more complex enterprise supply chain environments. A multi-location risk classification system could also incorporate inter-warehouse transfer recommendations to rebalance stock and reduce risk across the network.

Automate the Dashboard with Python Integration

The current Excel dashboard requires manual updates when new data becomes available. Integrating the dashboard generation process with the Python machine learning pipeline using libraries such as openpyxl or xlsxwriter would enable fully automated dashboard refresh whenever new inventory data is processed. This would eliminate manual effort, reduce the risk of human error in dashboard updates, and ensure that decision-makers always have access to the most current risk classification results without any manual intervention.